



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΥΠΟΛΟΓΙΣΤΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

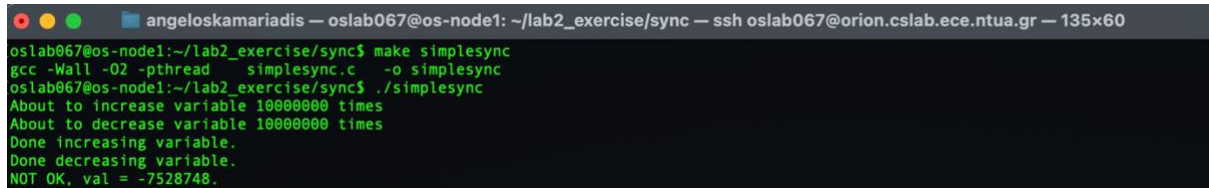
ΛΕΙΤΟΥΡΓΙΚΑ ΣΥΣΤΗΜΑΤΑ ΥΠΟΛΟΓΙΣΤΩΝ
2^η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ
ΣΥΓΧΡΟΝΙΣΜΟΣ

ΑΓΓΕΛΟΣ ΚΑΜΑΡΙΑΔΗΣ | ΠΑΝΑΓΙΩΤΗΣ ΖΩΓΡΑΦΟΣ

ΜΕΡΟΣ 1 - ΣΥΓΧΡΟΝΙΣΜΟΣ ΣΕ ΥΠΑΡΧΟΝΤΑ ΚΩΔΙΚΑ

Ζητούμενο 1 - Μεταγλώττιση του `simplesync.c`, με τη χρήση του `Makefile`

Αφού μεταγλωττίσαμε το αρχείο `simplesync.c` με τη διαδικασία που φαίνεται παρακάτω, παρατηρήσαμε πως ενώ η λογική επιβάλλει πως εάν ξεκινήσουμε από το 0, και προσθέσουμε το 1 N φορές, ενώ παράλληλα αφαιρέσουμε το 1 N φορές, το τελικό αποτέλεσμα της `val` θα έπρεπε να είναι πάλι 0, όμως στην πράξη, όταν τρέξαμε το πρόγραμμα για μεγάλο N, το αποτέλεσμα ήταν τυχαίο και διάφορο του μηδενός, ενώ άλλαζε σε κάθε εκτέλεση.



```
angeloskamariadis — oslab067@os-node1: ~/lab2_exercise/sync — ssh oslab067@orion.cslab.ece.ntua.gr — 135x60
oslab067@os-node1:~/lab2_exercise/sync$ make simplesync
gcc -Wall -O2 -pthread  simplesync.c  -o simplesync
oslab067@os-node1:~/lab2_exercise/sync$ ./simplesync
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
NOT OK, val = -7528748.
```

Αυτό συμβαίνει, διότι τα δύο threads διαβάζουν και γράφουν στην ίδια μεταβλητή (`val`) ταυτόχρονα, χωρίς καμία προστασία. Έτσι, ο scheduler του λειτουργικού συστήματος τα διακόπτει απρόβλεπτα, οδηγώντας σε lost updates.

Ζητούμενα 2,3 και 4 - Επέκταση του κώδικα του `simplesync.c` για συγχρονισμό με `POSIX mutexes` και `atomic` λειτουργίες

Αρχικά, επεκτείναμε τον κώδικα του αρχείου `simplesync.c` δύο φορές. Η πρώτη επέκταση αφορά τον συγχρονισμό της εκτέλεσης των threads με τη χρήση `mutexes`, ενώ η δεύτερη αφορά τον συγχρονισμό της εκτέλεσης των threads με τη χρήση ατομικών λειτουργιών.

Ο κώδικας του αρχείου `simplesync-mutex.c` απεικονίζεται παρακάτω.

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

#define perror_pthread(ret, msg) \
    do { errno = ret; perror(msg); } while (0)

#define N 10000000

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increase_fn(void *arg)
{
    int i;
    volatile int *ip = arg;

    fprintf(stderr, "About to increase variable %d times\n", N);
    for (i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        ++(*ip);
        pthread_mutex_unlock(&lock);
    }
    fprintf(stderr, "Done increasing variable.\n");

    return NULL;
}

void *decrease_fn(void *arg)
{
    int i;
    volatile int *ip = arg;

    fprintf(stderr, "About to decrease variable %d times\n", N);
    for (i = 0; i < N; i++) {
        pthread_mutex_lock(&lock);
        --(*ip);
        pthread_mutex_unlock(&lock);
    }
    fprintf(stderr, "Done decreasing variable.\n");

    return NULL;
}

int main(int argc, char *argv[])
{
    int val, ret, ok;
    pthread_t t1, t2;

    val = 0;

    ret = pthread_create(&t1, NULL, increase_fn, &val);
    if (ret) {
        perror_pthread(ret, "pthread_create");
        exit(1);
    }

    ret = pthread_create(&t2, NULL, decrease_fn, &val);

    if (ret) {
        perror_pthread(ret, "pthread_create");
        exit(1);
    }

    ret = pthread_join(t1, NULL);
    if (ret)
        perror_pthread(ret, "pthread_join");

    ret = pthread_join(t2, NULL);
    if (ret)
        perror_pthread(ret, "pthread_join");

    ok = (val == 0);

    printf("%sOK, val = %d.\n", ok ? "" : "NOT ", val);

    return ok;
}
```

1,1

Go

78.1

Go

Ο κώδικας του αρχείου simplesync-atomic.c απεικονίζεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab2_exercise/sync — ssh oslab067@orion.cslab.ece.ntua.gr — 116...
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>

#define perror_pthread(ret, msg) \
    do { errno = ret; perror(msg); } while (0)

#define N 10000000

void *increase_fn(void *arg)
{
    int i;
    volatile int *ip = arg;

    fprintf(stderr, "About to increase variable %d times\n", N);
    for (i = 0; i < N; i++) {
        __sync_fetch_and_add(ip, 1);
    }
    fprintf(stderr, "Done increasing variable.\n");

    return NULL;
}

void *decrease_fn(void *arg)
{
    int i;
    volatile int *ip = arg;

    fprintf(stderr, "About to decrease variable %d times\n", N);
    for (i = 0; i < N; i++) {
        __sync_fetch_and_sub(ip, 1);
    }
    fprintf(stderr, "Done decreasing variable.\n");

    return NULL;
}

int main(int argc, char *argv[])
{
    int val, ret, ok;
    pthread_t t1, t2;

    val = 0;

    ret = pthread_create(&t1, NULL, increase_fn, &val);
    if (ret) {
        perror_pthread(ret, "pthread_create");
        exit(1);
    }

    ret = pthread_create(&t2, NULL, decrease_fn, &val);
    if (ret) {
        perror_pthread(ret, "pthread_create");
        exit(1);
    }

    ret = pthread_join(t1, NULL);
    if (ret)
        perror_pthread(ret, "pthread_join");

    ret = pthread_join(t2, NULL);
    if (ret)
        perror_pthread(ret, "pthread_join");

    ok = (val == 0);

    printf("%sOK, val = %d.\n", ok ? "" : "NOT ", val);

    return ok;
}
```

Η ορθή λειτουργίων των δύο αρχείων κώδικα, επιβεβαιώνεται από τις παρακάτω εικόνες.

```
angeloskamariadis — oslab067@os-node1: ~/lab2_exercise/sync — ssh oslab067@orion.cslab.ece.ntua.gr — 135x60
oslab067@os-node1:~/lab2_exercise/sync$ make simplesync-mutex
gcc -Wall -O2 -pthread -DSYNC_MUTEX -c -o simplesync-mutex.o simplesync.c
gcc -Wall -O2 -pthread -o simplesync-mutex simplesync-mutex.o
oslab067@os-node1:~/lab2_exercise/sync$ ./simplesync-mutex
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
OK, val = 0.

angeloskamariadis — oslab067@os-node1: ~/lab2_exercise/sync — ssh oslab067@orion.cslab.ece.ntua.gr — 135x60
oslab067@os-node1:~/lab2_exercise/sync$ make simplesync-atomic
gcc -Wall -O2 -pthread -DSYNC_ATOMIC -c -o simplesync-atomic.o simplesync.c
gcc -Wall -O2 -pthread -o simplesync-atomic simplesync-atomic.o
oslab067@os-node1:~/lab2_exercise/sync$ ./simplesync-atomic
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
OK, val = 0.
```

Όπως ξέρουμε από τον κώδικα, η κοινόχρηστη μεταβλητή `val` ξεκινά από το 0 και αυξάνεται 10 εκατομμύρια φορές από το πρώτο νήμα, ενώ μειώνεται άλλες τόσες από το δεύτερο. Χωρίς μηχανισμούς συγχρονισμού, η ταυτόχρονη αυτή προσπάθεια προκαλεί `race`, με αποτέλεσμα να γράφει το ένα νήμα πάνω στο άλλο, να χάνονται πράξεις και η τελική τιμή να είναι τυχαία. Η χρήση των `Mutexes` ή των `Atomics` διασφαλίζει ότι κάθε αυξομειώση εκτελείται αποκλειστικά και προστατευμένα ως ένα αδιαίρετο βήμα. Επομένως, το γεγονός ότι το πρόγραμμα τερματίζει ακριβώς με `val = 0` αποτελεί την πρακτική απόδειξη ότι δεν χάθηκε απολύτως καμία ενημέρωση της μνήμης και ο συγχρονισμός λειτούργησε άψογα.

Ερωτήσεις

- Ερώτηση 1

Χρησιμοποιώντας την εντολή `time(1)`, μετρήσαμε τον χρόνο εκτέλεσης των τριών εκτελέσιμων αρχείων. Έτσι, είμασταν σε θέση να συγκρίνουμε τον χρόνο εκτέλεσης των εκτελέσιμων που εκτελούν συγχρονισμό (`simplesync-mutex` και `simplesync-atomic`), σε σχέση με το χρόνο εκτέλεσης του αρχικού προγράμματος (`simplesync`) χωρίς συγχρονισμό.

Η χρονική σύγκριση των των τριών αρχείων φαίνεται παρακάτω.

```
angeloskamariadis — oslab067@os-node1: ~/lab2_exercise/sync — ssh oslab067@orion.cslab.ece.ntua.gr — 147x60
oslab067@os-node1:~/lab2_exercise/sync$ time ./simplesync
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
NOT OK, val = -1381562.

real    0m0.575s
user    0m1.139s
sys     0m0.004s
oslab067@os-node1:~/lab2_exercise/sync$ time ./simplesync-mutex
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
OK, val = 0.

real    0m9.465s
user    0m10.620s
sys     0m7.952s
oslab067@os-node1:~/lab2_exercise/sync$ time ./simplesync-atomic
About to increase variable 10000000 times
About to decrease variable 10000000 times
Done increasing variable.
Done decreasing variable.
OK, val = 0.

real    0m2.307s
user    0m4.457s
sys     0m0.005s
```

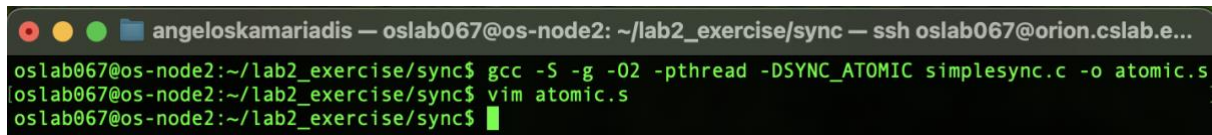
Από την πιο πάνω εικόνα, παρατηρούμε πως τα προγράμματα με συγχρονισμό είναι σημαντικά πιο αργά. Ο λόγος είναι ότι ο συγχρονισμός εισάγει overhead. Χωρίς συγχρονισμό, τα νήματα τρέχουν εντελώς παράλληλα, αξιοποιώντας στο έπακρο την CPU και την cache. Όταν βάζουμε συγχρονισμό, αναγκάζουμε το σύστημα να εκτελεί το κρίσιμο τμήμα (την αυξομείωση της μεταβλητής val) σειριακά. Επιπλέον, ο συγχρονισμός προκαλεί cache invalidations ανάμεσα στους πυρήνες του επεξεργαστή, καθώς ο ένας πυρήνας περιμένει τον άλλον να γράψει στη μνήμη.

▪ Ερώτηση 2

Η μέθοδος των ατομικών λειτουργιών, είναι σαφέστερα γρηγορότερη από τη χρήση POSIX mutexes. Ο λόγος είναι πως οι ατομικές λειτουργίες, δρουν στο επίπεδο του Hardware και υλοποιούνται απευθείας από τον ίδιο τον CPU. Έτσι, δεν απαιτούν την παρέμβαση του λειτουργικού συστήματος καθώς ο CPU απλώς κλειδώνει τον δίαυλο μνήμης για ένα κλάσμα του δευτερολέπτου για να κάνει την πράξη. Από την άλλη, τα mutexes δρουν στο επίπεδο του λογισμικού (OS), έτσι όταν ένα νήμα βρίσκει ένα mutex κλειδωμένο, το Λειτουργικό Σύστημα πρέπει να το βάλει σε κατάσταση sleep, να κάνει context switch σε άλλο νήμα, και αργότερα να το ξυπνήσει. Όλη αυτή η διαδικασία απαιτεί system calls, οι οποίες κοστίζουν χιλιάδες κύκλους ρολογιού, κάνοντας τον χρόνο εκτέλεσης σημαντικά μεγαλύτερο.

▪ Ερώτηση 3

Χρησιμοποιώντας την εντολή "gcc -S -g -O2 -pthread -DSYNC_ATOMIC simplesync.c -o atomic.s", δημιουργήθηκε ο ενδιάμεσος κώδικας Assembly τον οποίο μπορούμε να δούμε χρησιμοποιώντας την εντολή "vim atomic.s".

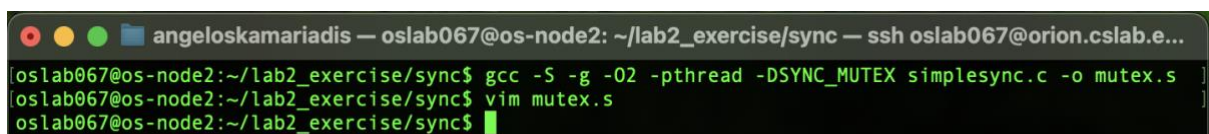


```
angeloskamariadis — oslab067@os-node2: ~/lab2_exercise/sync — ssh oslab067@orion.cs.cslab.e...
oslab067@os-node2:~/lab2_exercise/sync$ gcc -S -g -O2 -pthread -DSYNC_ATOMIC simplesync.c -o atomic.s
oslab067@os-node2:~/lab2_exercise/sync$ vim atomic.s
oslab067@os-node2:~/lab2_exercise/sync$
```

Λόγω του μεγέθους του, δεν τον παραθέτουμε στον εν λόγω αρχείο, όμως από τον κώδικα, προκύπτει πως στην αρχιτεκτονική x86_64, η συνάρτηση __sync_fetch_and_add μεταφράζεται στην εντολή Assembly lock addl ... ή lock xadd Το κρίσιμο στοιχείο εδώ, είναι το πρόθεμα lock. Αυτό δίνει εντολή στον επεξεργαστή να αποκτήσει αποκλειστική πρόσβαση στην κοινόχρηστη μνήμη για τη διάρκεια της εντολής πρόσθεσης/αφαίρεσης, κάνοντάς την ατομική σε επίπεδο hardware.

▪ Ερώτηση 4

Χρησιμοποιώντας την εντολή "gcc -S -g -O2 -pthread -DSYNC_MUTEX simplesync.c -o mutex.s", δημιουργήθηκε ο κώδικας Assembly τον οποίο μπορούμε να δούμε χρησιμοποιώντας την εντολή "vim mutex.s".



```
angeloskamariadis — oslab067@os-node2: ~/lab2_exercise/sync — ssh oslab067@orion.cs.cslab.e...
oslab067@os-node2:~/lab2_exercise/sync$ gcc -S -g -O2 -pthread -DSYNC_MUTEX simplesync.c -o mutex.s
oslab067@os-node2:~/lab2_exercise/sync$ vim mutex.s
oslab067@os-node2:~/lab2_exercise/sync$
```

Σε αντίθεση με τα Atomics που έγιναν μία εντολή CPU, το `pthread_mutex_lock()` δεν μεταφράζεται σε μία συγκεκριμένη εντολή του επεξεργαστή, αλλά σε μια function call. Στην Assembly, παρατηρούμε το "call pthread_mutex_lock@PLT". Αυτό, αποδεικνύει όσα αναφέραμε στο ερώτημα 2. Το πρόγραμμα σταματά τη ροή του, καλεί τη βιβλιοθήκη pthread η οποία με τη σειρά της εκτελεί πολύπλοκο κώδικα ελέγχου και αν χρειαστεί, καλεί τον πυρήνα του Λειτουργικού Συστήματος (kernel) για να μπλοκάρει το νήμα. Για αυτό προκαλείται η μεγαλύτερη καθυστέρηση στην εκτέλεση του κώδικα του αρχείου.

ΠΑΡΑΛΛΗΛΟΣ ΥΠΟΛΟΓΙΣΜΟΣ ΤΟΥ ΣΥΝΟΛΟΥ MANDELBROT

Ζητούμενο 1 - Συγχρονισμός με Semaphores

Η επέκταση του κώδικα του αρχείου mandel.c (mandel-sem.c) που υλοποιεί συγχρονισμό με semaphores φαίνεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab2_exercise/partB — ssh oslab067@orion.cslab.ec...
*
* mandel-sem.c
*
* Parallel Mandelbrot with POSIX semaphores.
*/

#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#include "mandel-lib.h"

#define MANDEL_MAX_ITERATION 100000

int y_chars = 50;
int x_chars = 90;

double xmin = -1.8, xmax = 1.0;
double ymin = -1.0, ymax = 1.0;

double xstep;
double ystep;

int NTHREADS;
sem_t *sems;

typedef struct {
    int tid;
} thread_arg_t;

void compute_mandel_line(int line, int color_val[])
{
    double x, y;
    int n;
    int val;

    y = ymax - ystep * line;

    for (x = xmin, n = 0; n < x_chars; x += xstep, n++) {
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);

        if (val > 255)
            val = 255;

        val = xterm_color(val);
        color_val[n] = val;
    }
}

void output_mandel_line(int fd, int color_val[])
{
    int i;
    char point = '@';
```

```

char newline = '\n';

for (i = 0; i < x_chars; i++) {
    set_xterm_color(fd, color_val[i]);

    if (write(fd, &point, 1) != 1) {
        perror("output_mandel_line: write point");
        exit(1);
    }
}

if (write(fd, &newline, 1) != 1) {
    perror("output_mandel_line: write newline");
    exit(1);
}
}

void *thread_func(void *arg)
{
    thread_arg_t *targ = (thread_arg_t *)arg;
    int tid = targ->tid;

    for (int line = tid; line < y_chars; line += NTHREADS) {
        int color_val[x_chars];

        compute_mandel_line(line, color_val);

        sem_wait(&sems[tid]);

        output_mandel_line(STDOUT_FILENO, color_val);

        sem_post(&sems[(tid + 1) % NTHREADS]);
    }

    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t *threads;
    thread_arg_t *args;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s NTHREADS\n", argv[0]);
        exit(1);
    }

    NTHREADS = atoi(argv[1]);

    if (NTHREADS <= 0) {
        fprintf(stderr, "NTHREADS must be positive\n");
        exit(1);
    }

    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    threads = malloc(NTHREADS * sizeof(pthread_t));
    args = malloc(NTHREADS * sizeof(thread_arg_t));
    sems = malloc(NTHREADS * sizeof(sem_t));

    if (threads == NULL || args == NULL || sems == NULL) {
        perror("malloc");
        exit(1);
    }

    for (int i = 0; i < NTHREADS; i++) {
        sem_init(&sems[i], 0, i == 0 ? 1 : 0);
    }

    for (int i = 0; i < NTHREADS; i++) {
        args[i].tid = i;

        if (pthread_create(&threads[i], NULL, thread_func, &args[i]) != 0) {
            perror("pthread_create");
            exit(1);
        }
    }

    for (int i = 0; i < NTHREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    reset_xterm_color(STDOUT_FILENO);

    for (int i = 0; i < NTHREADS; i++) {
        sem_destroy(&sems[i]);
    }

    free(sems);
    free(args);
    free(threads);

    return 0;
}

```

Ο άνωθεν κώδικας, δίνει το πιο κάτω output, το οποίο επιβεβαιώνει την ορθή λειτουργία του. Επίσης φαίνεται και ο χρόνος εκτέλεσης του.

[illegible]

Ζητούμενο 2 - Συγχρονισμός με *Condition Variables*

Η επέκταση του κώδικα του αρχείου mandel.c (mandel-cond.c) που υλοποιεί συγχρονισμό με condition variables φαίνεται παρακάτω.

```
angeloskamariadis — oslab067@os-node2: ~/lab2_exercise/partB — ssh oslab067@orion.cslab.ec...
/*
 * mandel-cond.c
 *
 * Parallel Mandelbrot with POSIX condition variables.
 */

#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <pthread.h>

#include "mandel-lib.h"

#define MANDEL_MAX_ITERATION 100000

int y_chars = 50;
int x_chars = 90;

double xmin = -1.8, xmax = 1.0;
double ymin = -1.0, ymax = 1.0;

double xstep;
double ystep;

int NTHREADS;
int turn = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t *conds;

typedef struct {
    int tid;
} thread_arg_t;

void compute_mandel_line(int line, int color_val[])
{
    double x, y;
    int n;
    int val;

    y = ymax - ystep * line;

    for (x = xmin, n = 0; n < x_chars; x += xstep, n++) {
        val = mandel_iterations_at_point(x, y, MANDEL_MAX_ITERATION);

        if (val > 255)
            val = 255;

        val = xterm_color(val);
        color_val[n] = val;
    }
}

void output_mandel_line(int fd, int color_val[])
{

```

```

angeloskamariadis — oslab067@os-node2: ~/lab2_exercise/partB — ssh oslab067@orion.cslab.ec...

int i;
char point = '@';
char newline = '\n';

for (i = 0; i < x_chars; i++) {
    set_xterm_color(fd, color_val[i]);

    if (write(fd, &point, 1) != 1) {
        perror("output_mandel_line: write point");
        exit(1);
    }

    if (write(fd, &newline, 1) != 1) {
        perror("output_mandel_line: write newline");
        exit(1);
    }
}

void *thread_func(void *arg)
{
    thread_arg_t *targ = (thread_arg_t *)arg;
    int tid = targ->tid;

    for (int line = tid; line < y_chars; line += NTHREADS) {
        int color_val[x_chars];

        compute_mandel_line(line, color_val);

        pthread_mutex_lock(&lock);

        while (turn != tid) {
            pthread_cond_wait(&conds[tid], &lock);
        }

        output_mandel_line(STDOUT_FILENO, color_val);

        turn = (turn + 1) % NTHREADS;

        pthread_cond_signal(&conds[turn]);

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main(int argc, char **argv)
{
    pthread_t *threads;
    thread_arg_t *args;

    if (argc != 2) {
        fprintf(stderr, "Usage: %s NTHREADS\n", argv[0]);
        exit(1);
    }

    NTHREADS = atoi(argv[1]);

    if (NTHREADS <= 0) {
        fprintf(stderr, "NTHREADS must be positive\n");
        exit(1);
    }

    xstep = (xmax - xmin) / x_chars;
    ystep = (ymax - ymin) / y_chars;

    threads = malloc(NTHREADS * sizeof(pthread_t));
    args = malloc(NTHREADS * sizeof(thread_arg_t));
    conds = malloc(NTHREADS * sizeof(pthread_cond_t));

    if (threads == NULL || args == NULL || conds == NULL) {
        perror("malloc");
        exit(1);
    }

    for (int i = 0; i < NTHREADS; i++) {
        pthread_cond_init(&conds[i], NULL);
    }

    for (int i = 0; i < NTHREADS; i++) {
        args[i].tid = i;

        if (pthread_create(&threads[i], NULL, thread_func, &args[i]) != 0) {
            perror("pthread_create");
            exit(1);
        }
    }

    for (int i = 0; i < NTHREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    reset_xterm_color(STDOUT_FILENO);

    for (int i = 0; i < NTHREADS; i++) {
        pthread_cond_destroy(&conds[i]);
    }

    pthread_mutex_destroy(&lock);

    free(conds);
    free(args);
    free(threads);

    return 0;
}

```

Ο άνωθεν κώδικας, δίνει το πιο κάτω output, το οποίο επιβεβαιώνει την ορθή λειτουργία του. Επίσης φαίνεται και ο χρόνος εκτέλεσης του.

Πρόγραμμα	Χρόνος
Serial ./mandel	2.989s
Semaphore ./mandel-sem 2	1.473s
Condition variables ./mandel-cond 2	1.331s

Όπως φαίνεται από την πιο πάνω εικόνα, το παράλληλο πρόγραμμα με semaphores είχε επιτάχυνση περίπου $2.989 / 1.473 = 2.03x$. Το πρόγραμμα με condition variables είχε επιτάχυνση περίπου $2.989 / 1.331 = 2.25x$. Η διαφορά πάνω από το ιδανικό $2x$ μπορεί να οφείλεται σε μικρές διακυμάνσεις μέτρησης, caching και στο ότι οι χρόνοι είναι σχετικά μικροί.

▪ Ερώτημα 3

Στη δεύτερη εκδοχή χρησιμοποιήθηκαν NTHREADS condition variables, μία για κάθε thread, μαζί με ένα mutex και μία κοινή μεταβλητή turn. Η μεταβλητή turn δείχνει ποιο thread έχει σειρά να τυπώσει. Κάθε thread περιμένει στη δική του condition variable μέχρι να ισχύσει `turn == tid`.

Για μία condition variable: Αν χρησιμοποιηθεί μόνο μία condition variable, όλα τα threads θα περιμένουν στο ίδιο condition variable. Τότε ο συγχρονισμός μπορεί να γίνει με κοινή μεταβλητή turn και `pthread_cond_broadcast`. Αυτό λειτουργεί, αλλά έχει πρόβλημα επίδοσης, γιατί ξυπνούν πολλά threads ενώ μόνο ένα έχει τελικά δικαίωμα να συνεχίσει. Άρα γίνονται άσκοπα wakeups και περιττό scheduling. Οι διαφάνειες τονίζουν ότι με condition variables πρέπει να ελέγχουμε τη συνθήκη με `while` και ότι το `broadcast` ξυπνά όλα τα threads που περιμένουν στο ίδιο condition variable.

▪ Ερώτημα 4

Ναι, στις μετρήσεις εμφανίζεται επιτάχυνση. Το σειριακό πρόγραμμα χρειάστηκε περίπου 2.989s, ενώ οι παράλληλες εκδόσεις με 2 threads χρειάστηκαν περίπου 1.473s και 1.331s. Αυτό δείχνει ότι μέρος του υπολογισμού γίνεται παράλληλα. Το κρίσιμο σημείο είναι το μέγεθος του κρίσιμου τμήματος. Αν μέσα στο κρίσιμο τμήμα βάζαμε και τον υπολογισμό και την εκτύπωση κάθε γραμμής, τότε τα threads θα εκτελούνταν σχεδόν σειριακά και δεν θα υπήρχε ουσιαστική επιτάχυνση. Για αυτό στη λύση ο υπολογισμός της γραμμής γίνεται εκτός κρίσιμου τμήματος, ενώ μέσα στο συγχρονισμένο τμήμα γίνεται μόνο η εκτύπωση, ώστε οι γραμμές να εμφανίζονται με σωστή σειρά. Αυτό απαντά ακριβώς στην υπόδειξη της εκφώνησης: αν το κρίσιμο τμήμα περιέχει και τη φάση υπολογισμού και τη φάση εξόδου, τότε περιορίζεται η παραλληλία.

▪ Ερώτημα 5

Αν πατηθεί `Ctrl-C` όσο εκτελείται το πρόγραμμα, το πρόγραμμα μπορεί να τερματιστεί πριν καλέσει τη `reset_xterm_color()`. Επειδή το πρόγραμμα αλλάζει το χρώμα του terminal για να τυπώσει το Mandelbrot, υπάρχει περίπτωση το terminal να μείνει με λάθος χρώμα. Η χειροκίνητη λύση είναι η εντολή `reset`. Για να διορθωθεί αυτό προγραμματιστικά, μπορούμε να εγκαταστήσουμε signal handler για το `SIGINT`. Όταν ο χρήστης πατήσει `Ctrl-C`, ο handler θα καλεί πρώτα `reset_xterm_color(STDOUT_FILENO)` και μετά θα τερματίζει το πρόγραμμα. Έτσι το terminal θα επανέρχεται στην κανονική του κατάσταση ακόμη και μετά από διακοπή.